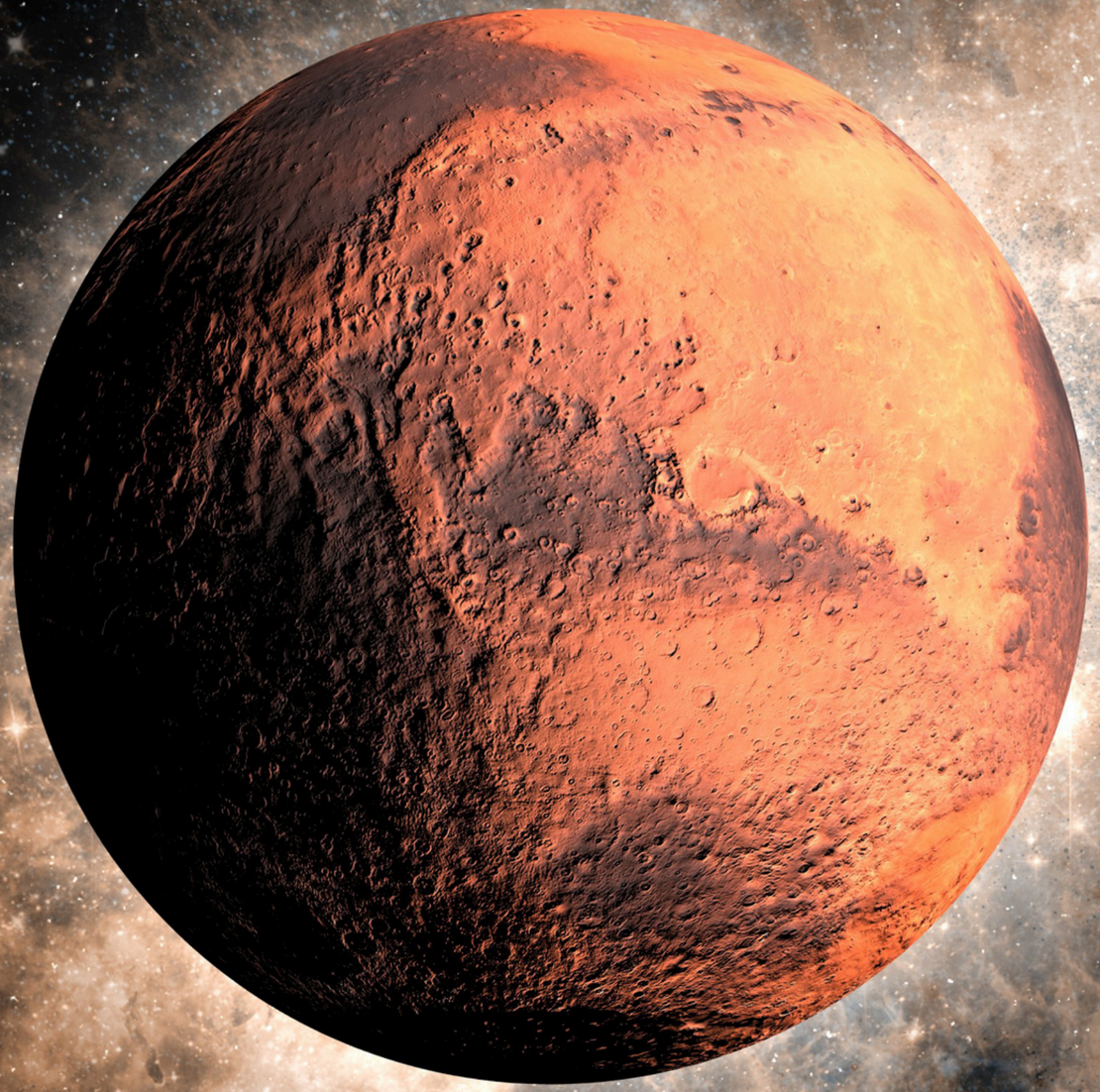


TERRAIN-AWARE AUTONOMOUS NAVIGATION ON MARS

ITAI2372–FINAL PRESENTATION

RAYYAAN HAAMID

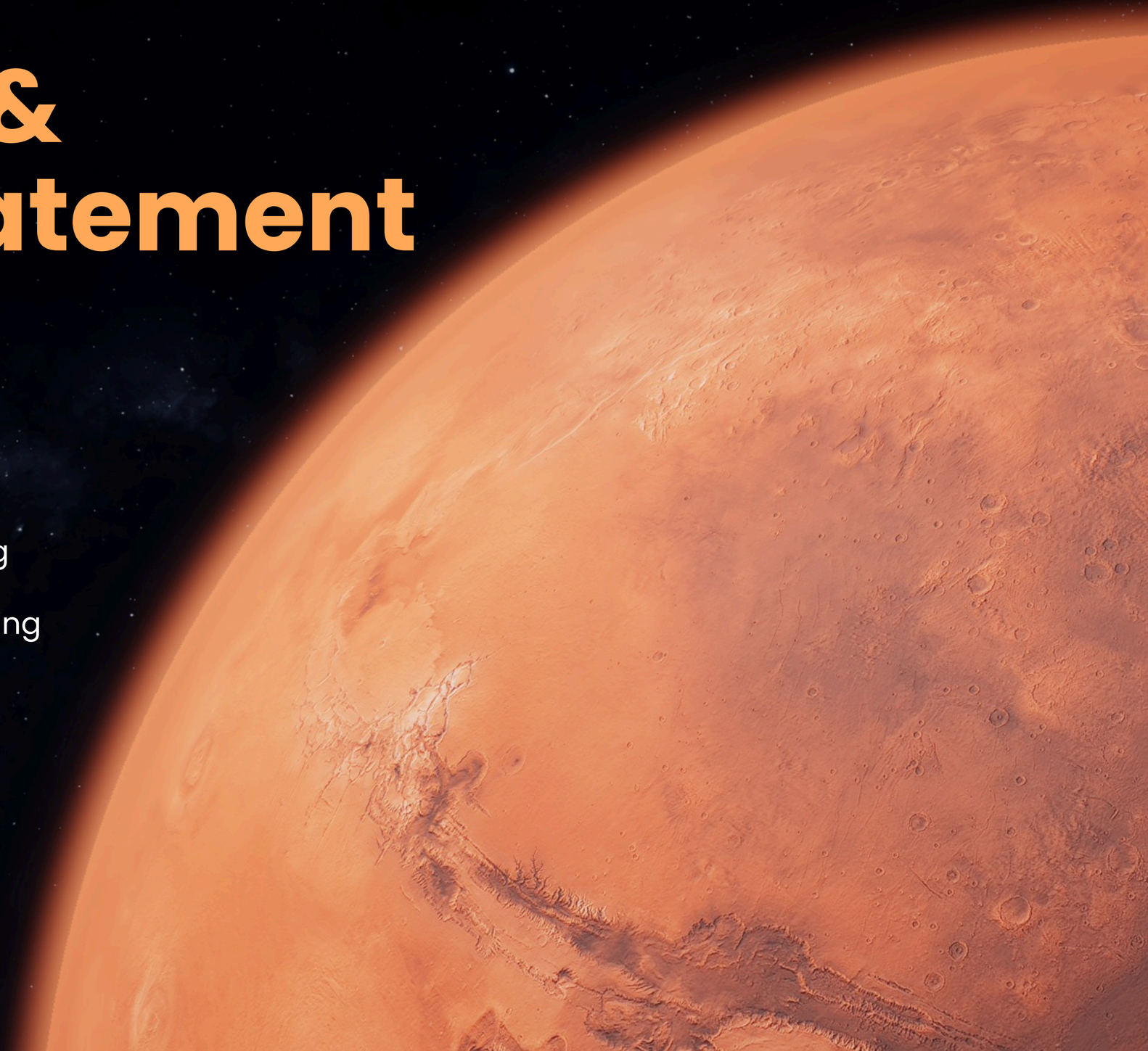
Motivation & Problem Statement

Why autonomous terrain perception is critical for Mars rovers

- Communication delay: 10–20 minutes round-trip → need on-board decision making
- Safety: Avoid hazards (large boulders, deep sand) to prevent mission-ending stuck or damage
- Efficiency: Select scientifically interesting targets while minimizing time lost to obstacles

Our goal:

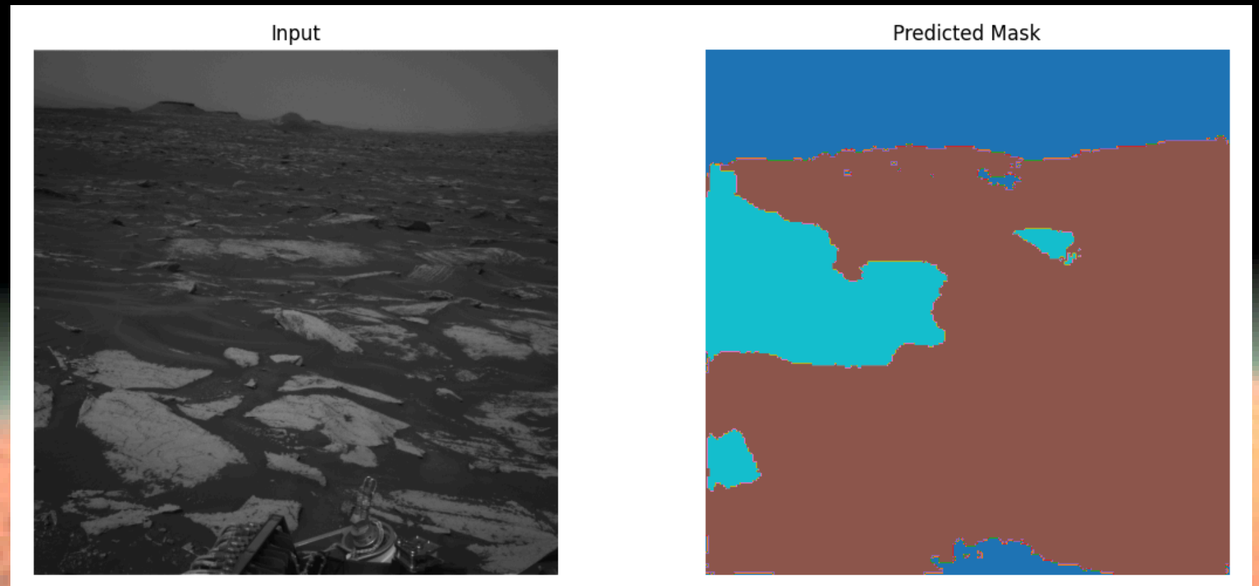
- Build a fast, accurate semantic-segmentation model to classify each pixel into
  a. Sand
  b. Bedrock
  c. Soil
  d. Big rock



DATASET OVERVIEW

AI4Mars MSL terrain dataset

- Source: Curiosity rover imagery, EDR format
- Total images: ~16 000 with hand-annotated masks
- Classes (4): sand, bedrock, soil, big rock
- Train/Val split: we subsampled 3 000 images → 80% train (2 400), 20% val (600)
- Mask format: 8-bit PNG, void pixels marked 255



DATA PREPARATION & AUGMENTATION

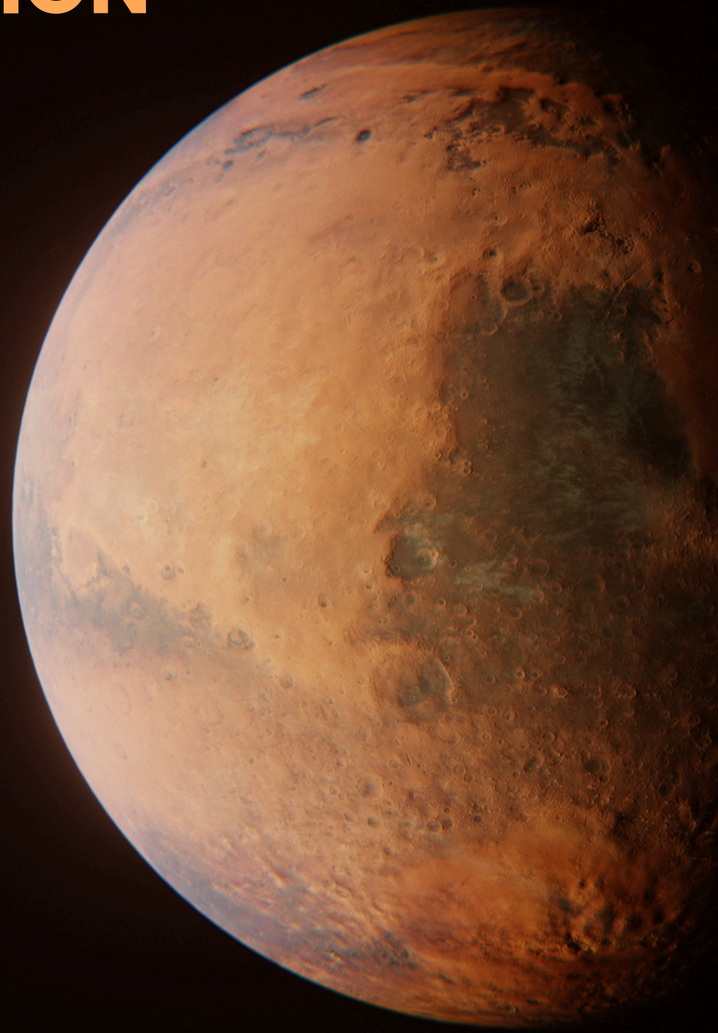
Pairing & splitting

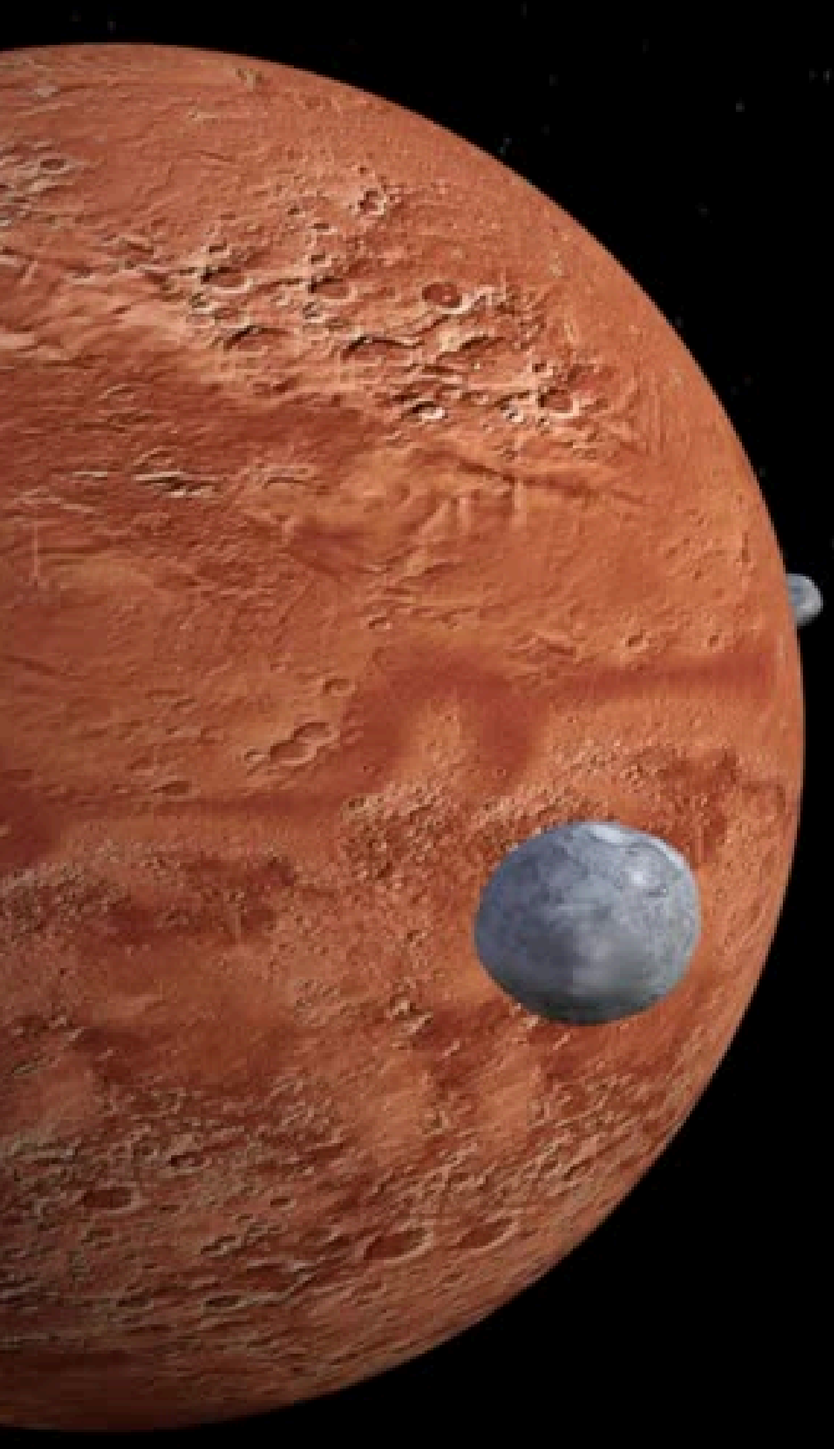
1. **Glob all mask PNGs → match to corresponding JPG/JPEG**
2. **Warn & drop any missing pairs**
3. **Shuffle + subsample 3 000**
4. **80/20 train/val split**

Augmentations (Albumentations)

- **Resize to 256×256**
- **RandomFlip (50%) & RandomRotate90 (50%)**
- **Affine (±15° rotation, 0.9–1.1 scale)**
- **Brightness/Contrast jitter (50%)**
- **Normalize (ImageNet mean/std)**
- **ToTensorV2 → torch.Tensor**

These increase robustness to lighting, orientation, scale.





MODEL ARCHITECTURE

U-Net w/ EfficientNet-B0 encoder

- Encoder: EfficientNet-B0 (pretrained on ImageNet)
- Decoder: symmetric upsampling with skip connections
- Head: 1×1 conv → 4 logits per pixel
- Parameters: ~5 million total

Why this design?

- Lightweight: fits 16 GB GPU at batch 4
- Proven: U-Net excels on sparse labels
- Transfer learning: ImageNet features accelerate convergence

Architecture diagram here: show encoder blocks, skip connections, decoder blocks

TRAINING SETUP & HYPERPARAMETERS

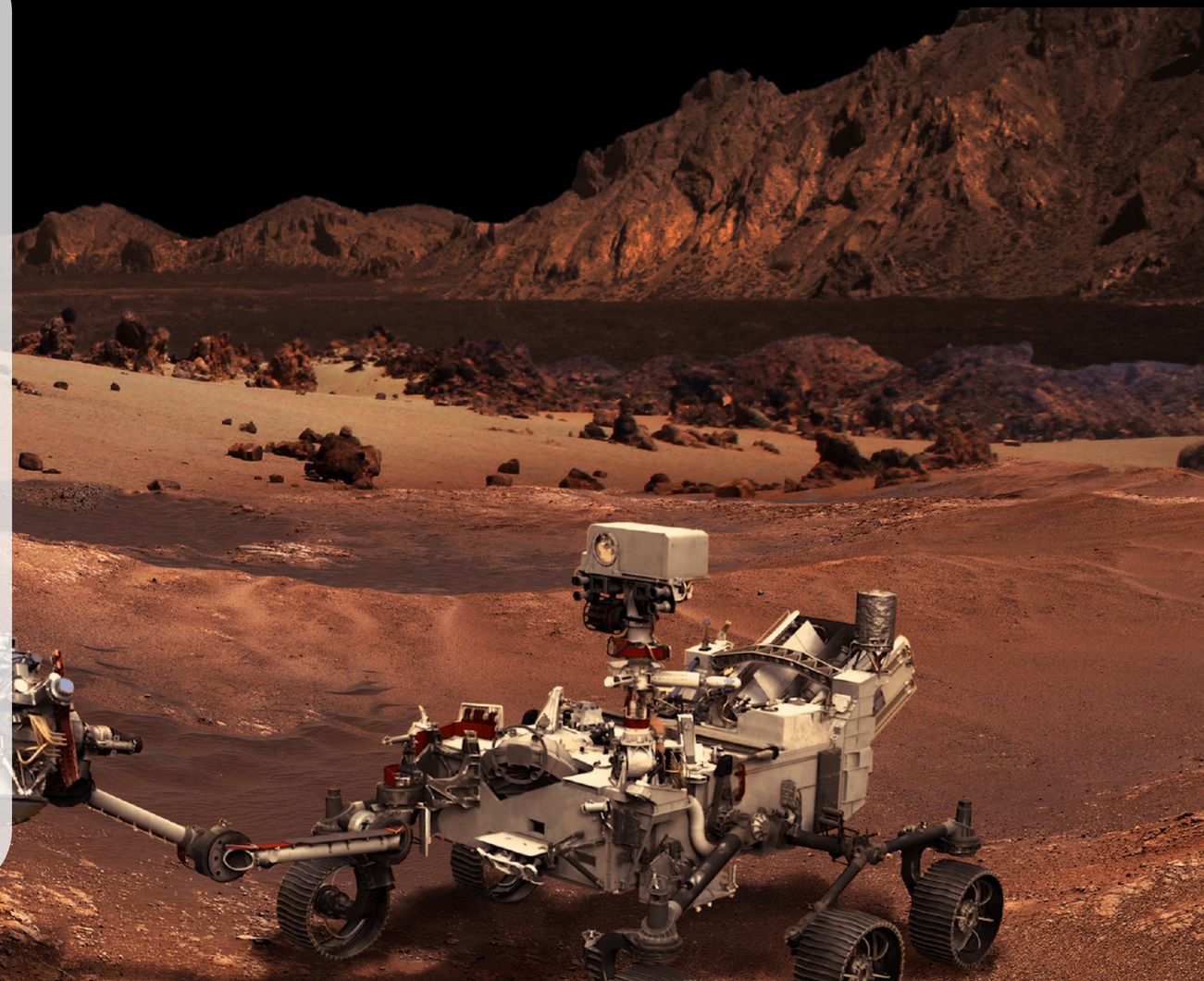
- Loss: Cross-entropy (ignore void index=255)
- Optimizer: AdamW, lr = 1e-4, weight_decay = 1e-2
- Batch size: 4
- Epochs: 20
- Hardware: NVIDIA Tesla T4 (16 GB)

Metrics logged per epoch:

- Train/Val Loss
- Mean IoU
- Pixel Accuracy

Final Val Results:

val_loss ≈ 0.14
mean IoU ≈ 0.85
pixel acc ≈ 0.90

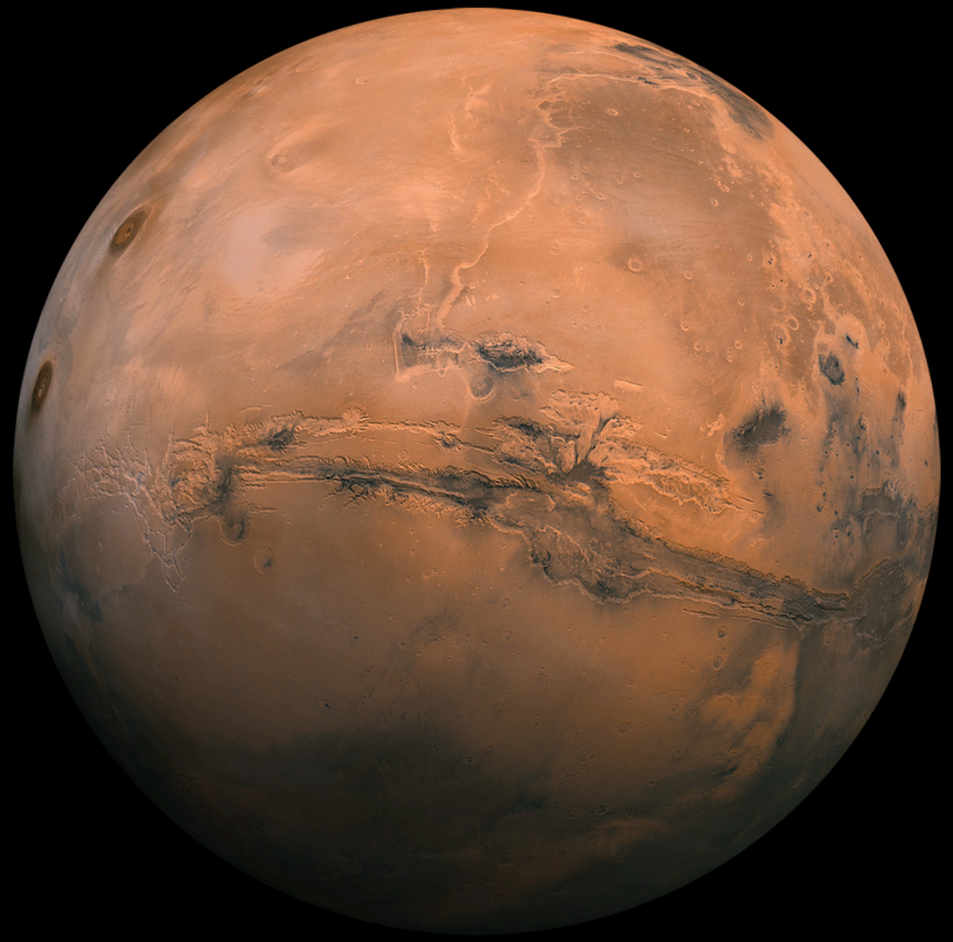


QUANTITATIVE RESULTS

Loss & IoU curves

- Rapid drop in first 5 epochs
- Plateau by epoch 15
- No overfitting: train & val curves track closely

Epoch	Train Loss	Val Loss	Val Mean IoU
1	0.7	0.29	0.62
10	0.24	0.17	0.83
20	0.17	0.14	0.85



QUALITATIVE RESULTS

Random samples from validation
For each row:

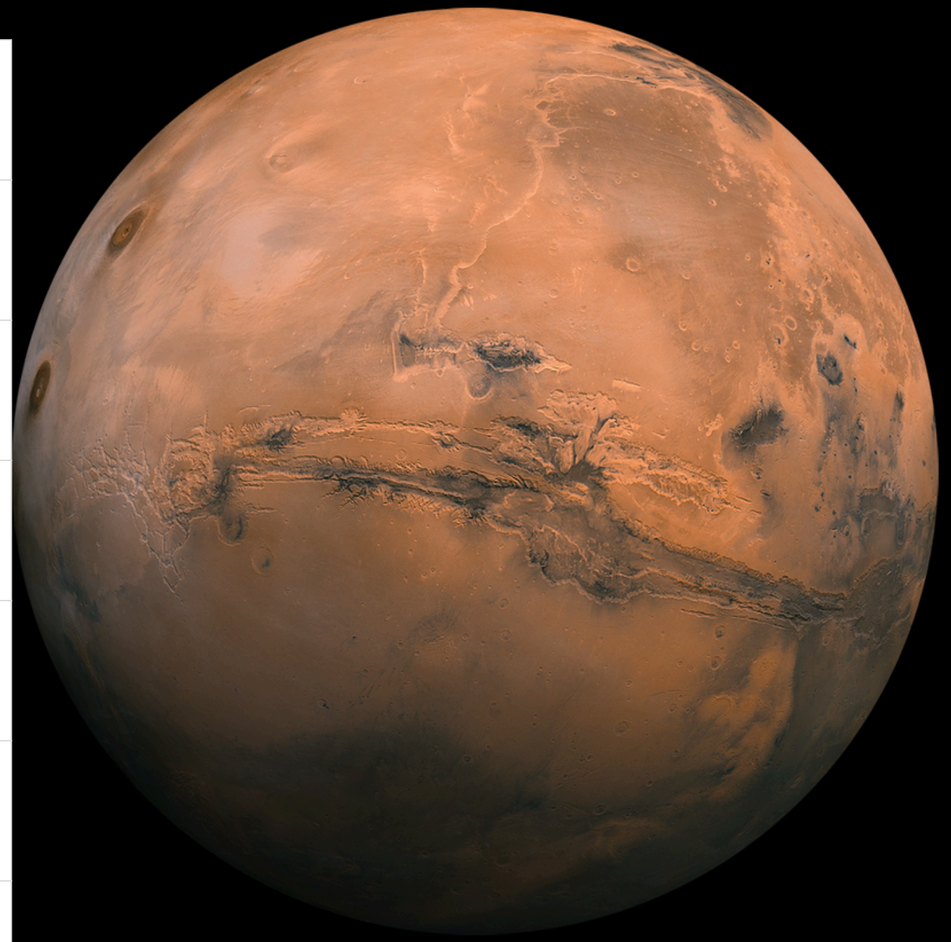
1. Left: original image
2. Center: ground-truth mask (colors = classes)
3. Right: predicted mask

Observations:

- Sand vs. soil: sometimes confusion along edges
- Bedrock detection: highly accurate
- Big rock: > 90% correct, few misses in shadow

CHALLENGES & SOLUTIONS

Challenge	Solution
TF-SegFormer Mismatch & TF errors	Switched to PyTorch & segmentation_models_pytorch
Albumentations API changes (Flip, Cutout)	Updated to HorizontalFlip, Affine, etc.
GPU OOM on full 16 k images	Subsampled 3 000 & batch size 4
Lack of accuracy logging	Added MeanIoU & pixel-accuracy metrics
Empty inference set (no test masks)	Random sampling of --n_samples images
Deployment format	Export via TorchScript + simple CLI script



FUTURE WORK & EXTENSIONS

- Better loss: incorporate focal + Dice to handle class imbalance
- Larger backbone: EfficientNet-B3 or SegFormer for higher accuracy
- Mixed precision: speed up training with AMP
- Edge deployment: optimize model for on-rover inference (TensorRT, ONNX)
- Continual learning: update with new rover data in flight